

Resolução - Classificação com Naive Bayes

Contexto: Uma instituição financeira deseja avaliar o risco de concessão de crédito calculando a probabilidade de um cliente se tornar inadimplente com base em características do seu perfil histórico.

Variáveis:

- y: inadimplente (1 = Sim; 0 = Não)
- x1: nome negativado em órgãos de crédito (1 = Sim; 0 = Não)
- x2: renda mensal comprovada estável (1 = Sim; 0 = Não)
- x3: possui garantias ou bens (1 = Sim; 0 = Não)

Dados históricos de treinamento:

Cliente	y	x1	x2	x3
1	1	1	0	0
2	1	1	1	0
3	1	0	0	0
4	1	1	0	1
5	1	1	0	0
6	0	0	1	1
7	0	1	1	0
8	0	0	1	0

1) Probabilidades a priori

$$P(y = 1) = 5/8 = 0.625$$

$$P(y = 0) = 3/8 = 0.375$$

2) Frequências e probabilidades condicionais

Classe	P(x1=1 y)	P(x2=1 y)	P(x3=1 y)	P(x1=0 y)	P(x2=0 y)	P(x3=0 y)
y = 1	0.800	0.200	0.200	0.200	0.800	0.800
y = 0	0.333	1.000	0.333	0.667	0.000	0.667

3) Regra do classificador Naive Bayes

Para um cliente com características $x = (x_1, x_2, x_3)$, calculamos:

$$P(y = 1 | x) \propto P(y = 1) \times P(x_1|y = 1) \times P(x_2|y = 1) \times P(x_3|y = 1)$$

$$P(y = 0 | x) \propto P(y = 0) \times P(x_1|y = 0) \times P(x_2|y = 0) \times P(x_3|y = 0)$$

4) Exemplo de classificação

Considere um cliente com $x_1 = 1, x_2 = 0, x_3 = 0$.

$$P(y = 1 | x) \propto 0.625 \times 0.8 \times 0.8 \times 0.8 = 0.320$$

$$P(y = 0 | x) \propto 0.375 \times (1/3) \times 0 \times (2/3) = 0.000$$

Como o valor de $P(y = 1 | x)$ é muito maior, o cliente é classificado como **inadimplente**.

5) Implementação em Python

```
import math

# Dados de treinamento
training = [
    {'y': 1, 'x1': 1, 'x2': 0, 'x3': 0},
    {'y': 1, 'x1': 1, 'x2': 1, 'x3': 0},
    {'y': 1, 'x1': 0, 'x2': 0, 'x3': 0},
    {'y': 1, 'x1': 1, 'x2': 0, 'x3': 1},
    {'y': 1, 'x1': 1, 'x2': 0, 'x3': 0},
    {'y': 0, 'x1': 0, 'x2': 1, 'x3': 1},
    {'y': 0, 'x1': 1, 'x2': 1, 'x3': 0},
    {'y': 0, 'x1': 0, 'x2': 1, 'x3': 0},
]

# Saber as classes
classes = sorted({row['y'] for row in training})

# Frequências condicionais
cond = {cls: {} for cls in classes}
for cls in classes:
    rows = [r for r in training if r['y'] == cls]
    for feature in ['x1', 'x2', 'x3']:
        cond[cls][feature] = {}
        for value in [0, 1]:
            cond[cls][feature][value] = sum(1 for r in rows if r[feature] == value) / len(rows)

# Probabilidades a priori
prior = {cls: sum(1 for r in training if r['y'] == cls) / len(training) for cls in classes}

def predict(cliente):
    scores = {}
    for cls in classes:
        score = prior[cls]
        for feature in ['x1', 'x2', 'x3']:
            score *= cond[cls][feature][cliente[feature]]
        scores[cls] = score
    total = sum(scores.values())
    return {cls: score / total for cls, score in scores.items()}

cliente = {'x1': 1, 'x2': 0, 'x3': 0}
print(predict(cliente))
```